

Cover Letter for TOSEM Submission

TyFlow: A Type-Aware Approach to Neural Code Models

Manuscript TOSEM-2026-0076 • Major Revision, decision of June 16, 2026

We thank the reviewers and the editor for their valuable comments. We have revised our paper accordingly. Every reviewer comment is quoted below in italics, followed by our response and by a summary of the changes we made in the paper; section and page numbers refer to the revised manuscript (45 pages).

Summary of changes. The revision adds two Java benchmarks (HumanEval-Java and TransCoder-GFG) that enlarge the Java evaluation to 186 tasks, a statistical analysis with 95% intervals and exact paired tests (App. D), and a task-level failure taxonomy (App. C). It also adds runtime and pruning measurements of the constrained decoder (RQ2). The comparison with other generation methods now covers three decoder-only models of 7B–30B parameters (Table 3) and four reliability-oriented approaches (RQ3, Table 5). A new Limitations subsection (Sec. 6.3) discusses the encoder-decoder dependence, dynamically typed languages, and type-system expressiveness.

Response to the Editor

Editor’s comment.

All the reviewers agree that the paper investigates an important topic. However, reviewers also point out major concerns that need to be addressed and responded to, hence my recommendation for a Major Revision. In the following, I have provided some of the critical concerns raised by the reviewers.

- *Missing failure analysis*
- *Limited evaluation benchmarks*
- *Missing discussion on scalability*
- *Missing comparison with larger and more recent LLMs*
- *Missing comparison with state-of-the-art techniques*
- *Missing analysis of the system’s cost*

Details on the above points are in the reviews of the reviewers involved in the process. While preparing a Major Revision, please make sure to respond to all the other reviewers’ comments and questions, not just the points summarized above.

Response.

Thanks. Each of the six concerns is addressed with new experiments or new analyses, as summarized below:

- **Failure analysis.** Every test task of the three Java benchmarks is classified into four categories in App. C (p. 42–43, Table 8), with two representative cases; the dominant residual mode of TyFlow is well-typed but semantically wrong code. Detailed response: [Comment 1.6](#).
- **Benchmarks.** HumanEval-Java (146/16) and TransCoder-GFG (414/103) are added, so the Java evaluation spans 186 test tasks from three independent sources, analyzed jointly in App. D (p. 44). Detailed responses: [Comment 1.3](#) and [Comment 2.1](#).
- **Scalability.** RQ2 measures the pruning strength of each component, the beam-exhaustion behavior, and the per-step and per-task runtime (up to $2.74\times$ the unconstrained base); scaling to richer type systems is discussed in the Limitations subsection. Detailed responses: [Comment 1.2](#) and [Comment 2.2](#).
- **Larger and more recent LLMs.** Zero- and few-shot pass@1 results for MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks are added (Table 3, p. 26): on Java they outperform the fine-tuned 2B models, on SuFu none solves a single task zero-shot. Detailed responses: [Comment 1.5](#) and [Comment 2.3](#).
- **State-of-the-art techniques.** RQ3 is rewritten as a five-method comparison on MBJP (rejection sampling, SynCode, Repilot, iterative compiler repair) on one leak-clean frozen checkpoint; none of the four external methods significantly improves functional correctness, while TyFlow-2B is the best of all compared methods. Detailed responses: [Comment 3.1](#) and [Comment 3.2](#).
- **System cost.** The runtime analysis is reported in RQ2 together with the component ablation it belongs to: per-step and per-task wall time of the same 220M model in the four configurations, with an explanation of where each cost comes from and why syntactic pruning even *lowers* the per-task total. For the compared external methods, the RQ3 setup states each method’s computational budget. Detailed responses: [Comment 2.2](#) and [Comment 3.3](#).

Beyond these six cross-cutting concerns, we respond to every other reviewer comment and question individually in Comments 1.1–3.6 below.

Response to Reviewer 1

We thank the reviewer for the positive assessment of the formalization, its generality over grammar-based approaches, and the ablation study.

Comment 1.1.

The example language ($\lambda \rightarrow$) is extremely simple. While it serves the pedagogical purpose well, readers may reasonably wonder whether the approach scales to languages with subtyping, polymorphism, type inference, overloading, or mutable state. The paper partially addresses this by evaluating on Java, but the overview could better acknowledge the gap between $\lambda \rightarrow$ and real languages.

The restriction to first-order unification deserves further discussion. Higher-order unification arises naturally in languages with higher-order functions and polymorphism. The authors should discuss what happens when this restriction is violated and what classes of type systems are excluded.

[Weaknesses:] The restriction to first-order unification and encoder-decoder architectures limits near-term applicability.

Response.

Thanks. We answer the three questions separately.

(1) The running example and richer type systems.

The difference from real languages. The example language is deliberately simple and is used only to demonstrate how TyFlow converts typing rules into synthesis rules. It contains Boolean and function types with explicitly annotated, monomorphic bindings. Unlike mainstream languages, it does not include features like subtyping, polymorphism, type inference, overloading, or mutable state. We now acknowledge this gap explicitly in the overview.

Extension to richer type systems. TyFlow is not tied to this example language. To support a richer statically typed language, we define its program, type, and context terms, express its typing rules as constrained Horn clauses, and implement the required computable constraints. The transformation from typing rules to synthesis rules remains unchanged. Because this language-specific implementation requires substantial engineering effort, we currently instantiate TyFlow only for Java and SuFu.

(2) The first-order unification restriction.

Why we use it. To apply a typing rule, TyFlow needs unification to terminate, decide whether the rule conclusion matches the current goal, and return a principal substitution for the remaining premises. First-order unification provides these guarantees: it is decidable and returns a most-general unifier that is unique up to variable renaming. We therefore use first-order unification as a sufficient condition.

Higher-order functions do not require higher-order unification. A higher-order function takes or returns a function value, and the type checker needs to determine the type of that value. Higher-order unification is needed only when the unknown metavariable to be solved in the typing rule is itself a function, type constructor, or predicate. The following two terms show the difference:

$$\text{Arrow}(\alpha, \beta) \quad \text{versus} \quad F(\text{Int}) = \text{List}(\text{Int}).$$

In the first term, *Arrow* is fixed. Only α and β are unknown, so this is still a first-order problem. In the second term, *F* is such an unknown, so it is a higher-order unification problem.

ML is an example. It supports higher-order functions and parametric polymorphism, but Hindley–Milner type inference uses first-order unification. Java provides the same distinction. A simplified typing rule for a higher-order API such as `Stream.map` is:

$$\frac{\Gamma \vdash s : \text{Stream}\langle A \rangle \quad \Gamma \vdash f : \text{Function}\langle A, B \rangle}{\Gamma \vdash s.\text{map}(f) : \text{Stream}\langle B \rangle}.$$

The program passes a function as a value, but `Stream` and `Function` are fixed type constructors. Rule application only needs to instantiate the ordinary type terms *A* and *B*. Thus, this higher-order language feature still requires only first-order unification.

Support for real-world type systems. Java supports function values, but its type checking does not require unification over unknown type-level functions, type constructors, or predicates. Its program, type, and context syntax can be represented as first-order terms. The Java Language Specification handles generic inference through constraint reduction, incorporation, and resolution over inference variables.¹ Features such as subtyping, generic bounds, overload applicability, and lambda target typing can therefore be handled through typing rules and language-specific constraints. Supporting these features requires substantial implementation work, but it does not require higher-order unification as long as rule application only instantiates ordinary term variables.

Dependent-type systems such as Lean are different because elaboration may need to infer an unknown type constructor or predicate. Lean therefore uses higher-order unification together with heuristics and backtracking.² The current TyFlow solver cannot directly support these cases. A decidable fragment with principal unifiers, such as higher-order patterns, could be incorporated, but this would require a new unifier. General higher-order unification remains outside our current framework.

(3) The architecture restriction. The current implementation cannot be moved directly because the encoder reads the changing synthesis goal. A decoder-only version would have to encode this goal at each step, and its cached states could not be reused. We discuss this issue and the decoder-only evaluation in [Comment 2.3](#).

¹*The Java Language Specification*, Java SE 25, Sec. 18, “Type Inference.”

²Leonardo de Moura, Jeremy Avigad, Soonho Kong, and Cody Roux. “Elaboration in Dependent Type Theory.” CADE-25, 2015.

Changes made.

- **Added** Sec. 2 (p. 9): the overview now enumerates the gap between $\lambda \rightarrow$ and real languages (subtyping, polymorphism, type inference, overloading resolution, mutable state) and points to the evaluation and the Limitations discussion.
- **Clarified** Sec. 3 (p. 12): why TyFlow uses first-order unification, why higher-order functions do not by themselves require higher-order unification, and when the current solver no longer applies.
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29): why the dynamic typing context is not directly portable to a decoder-only model.
- **Clarified** Sec. 6.3, “Type-system expressiveness” (p. 30): which features can use first-order terms and which require a different solver.

Revised manuscript, Sec. 2 (p. 9) – the acknowledged gap between the example language and real languages

The $\lambda \rightarrow$ language above serves for exposition: it is deliberately minimal, containing only the base type `bool`, function types, and explicitly annotated, monomorphic bindings. Real languages additionally involve subtyping, polymorphism, type inference, overloading resolution, and mutable state, none of which $\lambda \rightarrow$ exercises. The evaluation (Sec. 6) therefore runs TyFlow on Java and SuFu instances with fuller, though still restricted, type systems; the scope of the underlying rule framework is stated in Sec. 3, and how the approach would extend to richer type systems is discussed in Sec. 6.3.

Revised manuscript, Sec. 3 (p. 12) – the role and boundary of first-order unification

First-order unification always terminates. It either rejects two terms or returns a most-general unifier that is unique up to variable renaming. These guarantees let TyFlow apply a selected typing rule and update its remaining premises.

Supporting higher-order functions does not by itself require higher-order unification. The latter is needed only when an unknown to be solved is itself a function, type constructor, or predicate. If a typing rule creates such a problem, TyFlow’s current solver cannot handle it. A decidable fragment could be supported with a different unifier, but general higher-order unification remains outside the current framework.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29) – the architectural limitation

The representation itself carries over to a decoder-only model unchanged; what changes is the architecture, and with it the cost of the dynamic typing context. [...] so its key-value entries are recomputed at every step and never reused by later steps, and the computation is awkward to schedule. We leave this adaptation to future work.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – type-system expressiveness as a limitation

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. The current solver cannot handle a typing rule that requires higher-order unification, as discussed in Sec. 3.

Comment 1.2.

The practical implications of completeness deserve more discussion. Completeness says that for any well-typed program, there exists a synthesis derivation tree. But during generation, the LM must find the right sequence of decisions. If the LM makes a wrong choice early, it may be unable to recover. The paper does not discuss the branching factor of the search space or the likelihood of the LM making correct decisions in practice. Beam search partially addresses this, but the theoretical analysis gives no bounds on how many beams are needed.

The paper does not quantify the pruning rate in Section 5.3. What fraction of candidate tokens are eliminated at each level? This would help readers understand the practical search space reduction.

Response.

Thanks for this criticism. We now measure the constrained decoder, and we state explicitly what the completeness theorem does and does not imply for search.

(1) Completeness, branching factor, and beam behavior.

What completeness guarantees. The theorem is existential: it guarantees a derivation for every well-typed program, but it does not guarantee that the LM ranks a correct prefix within the beam, and it gives no bound on the required beam size. If all correct prefixes fall outside the beam, later decoding cannot recover them. Training on correct decision sequences is intended to assign higher probability to suitable choices.

Observed beam-10 behavior. We measure the effective branching factor as the number of active hypotheses present at each decoding step after pruning. At each step, the decoder applies grammar pruning, ranks the remaining expansions, and incrementally type-checks at most the 20 highest-scoring expansions to construct the next active beam. Theoretically, a finite beam can discard all correct prefixes even when the beam size is relatively large, so a subsequent active beam can become empty. However, this did not occur in the evaluated run where the beam width sets its maximum to 10: the active beam contained 7.97 hypotheses on average and never became empty. The average is lower than 10 because a completed program is moved to the output set, while rejected candidates do not fill an active slot.

(2) Pruning rates. Instrumenting the decoder on the SuFu tasks (beam 10, 330-step budget) gives: syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2% (type checking is applied only to the high-probability candidates about to enter the beam, which is why its rate is lower). A final, complete type check on finished derivations rejects 21.2% of the completed candidates (113/533).

Changes made.

- **Added** Sec. 6.2.2 (p. 27): “Whether the constraints over-prune”: what completeness does and does not guarantee, the theoretical possibility of beam exhaustion, and the observed beam-10 behavior.
- **Added** Sec. 6.2.2 (p. 27): “How much the constraints prune”: instrumented rates (59.1% syntactic, 8.2% type, 21.2% final check) and the 330-step budget.

Revised manuscript, Sec. 6.2.2, “Whether the constraints over-prune” (p. 27) – completeness and the observed beam-10 behavior

Completeness guarantees that a well-typed program has a derivation, but it does not guarantee that the model ranks a correct prefix within a fixed beam or provide a bound on the required beam size. A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [...] With beam 10, this number averages 7.97. [...] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.2.2, “How much the constraints prune” (p. 27) – the measured pruning rates

To measure the pruning strength, we instrument the constrained decoder on the SuFu tasks (beam 10, 330-step budget): syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2%. The lower rate is expected: type checking is more expensive, so it is applied only to the high-probability candidates about to enter the beam. Such candidates are rarely ill-typed, yet pruning them matters, for they would otherwise expand within the beam and crowd out well-typed derivations. Type checking during search is provisional (only completed AST fields are checked); the whole program is checked once when the derivation finishes, and this final check rejects 21.2% of the completed candidates.

Comment 1.3.

The SuFu test set has only ~58 tasks (20% of 290), and the Java test set has only ~61 tasks (10% of 608). These are very small test sets, and the results may not be statistically robust. The authors should report confidence intervals or conduct significance tests. A difference of 1–2 tasks passing or failing could shift pass@1 by several percentage points.

[Weaknesses:] *Small test sets raise concerns about statistical reliability. [...] Missing failure analysis and confidence intervals.*

Response.

Thanks for this comment. We address the dataset size and the statistical uncertainty separately.

(1) Expanded evaluation.

Benchmark coverage. We add two Java benchmarks from sources independent of MBJP. HumanEval-Java contains short algorithmic methods adapted from HumanEval, with a fixed split of 146 training and 16 test tasks. TransCoder-GFG contains numeric, string, and array problems drawn from the GeeksForGeeks programs released with TransCoder, with a fixed split of 414 training and 103 test tasks. Together with the original 67 MBJP test tasks, these

additions increase the Java evaluation from 67 to 186 test tasks across three benchmarks. SuFu remains a 58-task test set because the source repository contains 290 programs.

Experimental protocol. We evaluate the two added benchmarks at the 2B scale, comparing the baseline T5Gemma2-2B with our TyFlow-2B. Both systems use the same Java training tasks and are evaluated on the same fixed test tasks with beam 10 and ten candidates per task. We report pass@1, pass@10, FSP, and CER, so the evaluation covers functional correctness, candidate ranking, and compilation errors.

Results. The two added benchmarks show improvements on all four metrics:

- **HumanEval-Java.** pass@1 improves from 31.25% to 50.00%, pass@10 from 37.50% to 56.25%, and FSP from 6.50 to 4.75; CER falls from 24.38% to 1.30%.
- **TransCoder-GFG.** pass@1 improves from 19.42% to 30.10%, pass@10 from 33.01% to 46.60%, and FSP from 7.10 to 5.81; CER falls from 13.69% to 2.75%.

These results show that the improvements are not confined to the original MBJP split: they recur on two independently sourced Java benchmarks. The large CER reductions are consistent with the purpose of type-constrained generation, while the pass-rate and FSP gains show that the benefit is not limited to rejecting uncompileable outputs.

(2) Confidence intervals and significance tests. New App. D (p. 44) pairs every baseline–TyFlow run on the same tasks and reports a 95% interval and an exact two-sided paired test for every metric. For pass@1 and pass@10, we use Wilson intervals and McNemar tests; for FSP, a t -interval over per-task ranks and an exact sign test; and for CER, a Wilson interval over candidates and an exact sign test on per-task error fractions. The results show where the evidence is strong and where the small test sets still leave uncertainty:

- **Java.** At 220M, the Java evaluation contains only the 67 MBJP test tasks, and the small test set provides few paired observations: pass@10 improves from 20.90% to 28.36%, with 95% intervals of [12.88,32.07] for the baseline and [18.97,40.09] for TyFlow. The paired difference is not significant ($p = 0.359$), and only the CER difference is significant. At 2B, the two added benchmarks increase the Java test set to 186 paired tasks. On this larger test set, pass@10 improves from 33.33% to 46.24%, with 95% intervals of [26.96,40.38] and [39.22,53.41], respectively. The paired difference is significant ($p = 5.36 \times 10^{-4}$), as are the other three metric differences.
- **SuFu.** The small test set of 58 tasks also limits our ability to detect statistically significant improvements on this benchmark. At 2B, pass@10 improves from 37.93% to 48.28%, with 95% intervals of [26.56,50.80] for the baseline and [35.93,60.84] for TyFlow, but the paired difference is not significant ($p = 0.307$). At 220M, the pass@10 gain is larger: 32.76% to 53.45%. Its 95% intervals are [22.08,45.58] and [40.80,65.67], respectively, and the paired difference is significant ($p = 2.35 \times 10^{-3}$). All four paired differences are significant at 220M despite the same small test set.

Changes made.

- **Added** Sec. 6.1.1 (p. 24) and App. B (pp. 39–40): the sources, task characteristics, and fixed splits of HumanEval-Java and TransCoder-GFG.
- **Added** Sec. 6.2.1 (p. 25): the results of TyFlow-2B and its T5Gemma2-2B baseline on HumanEval-Java and TransCoder-GFG.
- **Added** App. D (pp. 44–45): 95% intervals and exact paired tests at the 220M and 2B scales (Tables 9–10), including the pooled 186-task Java analysis.

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added Java benchmarks and their fixed splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java [...], and TransCoder-GFG collects the Java programs released with TransCoder [...]; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, Sec. 6.2.1 (p. 25) – the results on the expanded benchmark set

TyFlow improves code generation accuracy at both scales and on every benchmark: [...] pass@1 rises [...] 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks. [...] TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture.

Revised manuscript, App. D (p. 44) – the paired-test definitions and the 220M outcomes

We pair every CodeT5-220M baseline run with the matching TyFlow-220M run on the same tasks, and compute exact two-sided McNemar tests on pass@1 / pass@10 and exact two-sided sign tests on FSP and on per-task compilation-error fractions (CER). pass@1 and pass@10 report Wilson 95% intervals over tasks, CER over candidates, and FSP a t -interval over per-task ranks [...] with ten candidates per task. In the SuFu block all four paired differences favor TyFlow at the 0.05 significance level. In the Java block only CER is significant at $n = 67$, for two reasons: the pass-rate improvements themselves are modest at this scale (one additional task at pass@1, five at pass@10), and a 67-task test set yields few discordant pairs (11, 19, and 24 for pass@1, pass@10, and FSP), so the exact tests are underpowered; pooling the three Java benchmarks at the 2B scale restores significance for all four metrics.

Revised manuscript, App. D (p. 44) – the pooled 186-task Java analysis

The three Java benchmarks (MBJP, HumanEval-Java, TransCoder-GFG) all retain per-task records, so we pool them into one combined 186-task analysis rather than reporting each benchmark separately; the same tests and intervals as at the 220M scale apply. In the combined analysis all four paired differences favor TyFlow at the 0.05 significance level (Tab. 10).

Revised manuscript, App. D (p. 44) – the 2B SuFu outcomes

On SuFu only the compilation-error difference is significant at the 2B scale: TyFlow-2B has zero compilation errors on all 313 rendered candidates, so every one of the 58 tasks favors TyFlow in the exact sign test. The pass@1, pass@10, and FSP differences favor TyFlow in the same direction as at the 220M scale but are not significant at $n = 58$: the gains are six solved tasks at both pass@1

and pass@10, which leaves few discordant pairs (8 vs. 14 for pass@1, 9 vs. 15 for pass@10) and 30 nonzero FSP pairs (18 better, 12 worse), so the exact tests are underpowered, mirroring the 220M Java block.

Comment 1.4.

On Java, TyFlow-220M improves pass@1 from 10.45% to only 11.94%, less than 1.5 percentage points absolute. While pass@10 improves more substantially, the pass@1 gain is marginal, given the test set size. The paper should discuss why Java gains are smaller.

[Weaknesses:] *The Java results are modest, and the subset restriction introduces potential selection bias.*

Response.

Thanks. We agree that the 1.5-percentage-point number is not convincing on its own. The revision responds in three ways: it explains the cause, it extends the Java evaluation, and it addresses the selection-bias concern.

(1) Why the 220M Java pass@1 gain is small. On SuFu the baseline starts from a low-resource language with many type errors, so the type constraints have many errors to fix. On Java the baseline has already been trained extensively on the language, so there is little room for improvement at this scale; what the constraints still remove is the uncompileable output, and that improvement appears in pass@10 (20.90% → 28.36%) and CER (35.52% → 1.53%).

(2) The gain grows with the model scale. The 220M figure comes from the smallest model on the smallest Java test set, so on its own it says little about what TyFlow achieves as capacity grows; at the 2B scale the same comparison yields much larger numbers, with pass@1 rising 13.43% → 25.37% (MBJP), 31.25% → 50.00% (HumanEval-Java), and 19.42% → 30.10% on TransCoder-GFG, with CER falling to 0.45%/1.30%/2.75%. The 220M figure is therefore the smallest-scale point of this trend, which the revision now documents.

(3) The selection-bias concern. This is a genuine scope limitation of the approach. The method requires programs to be expressible within the modeled type system and grammar: the toolchain admits about 78% of MBJP, and the two added benchmarks are similarly restricted to constructs the toolchain supports. The revision now states this explicitly in App. B, and the Limitations subsection (Sec. 6.3, p. 30) discusses why the restriction is practical rather than conceptual.

Changes made.

- **Added** Sec. 6.2.1 (p. 25): a paragraph explaining why the 220M MBJP pass@1 gain is the smallest.
- **Revised** Sec. 6.1.1 (p. 24): the MBJP protocol, now stated as 608 training and 67 held-out test tasks.
- **Added** App. B (p. 37): the statement that the Java-subset admission filter is a potential selection bias.

Revised manuscript, Sec. 6.2.1 (p. 25) – why the 220M Java pass@1 gain is the smallest

The 220M MBJP pass@1 gain is the smallest because there the type constraints mainly remove uncompileable candidates; the benefit shows up in pass@10 and CER instead, consistent with the failure analysis in Appendix C (well-typed but wrong is the dominant residual mode). TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture.

Revised manuscript, App. B (p. 37) – the selection-bias statement

The subset admits tasks whose constructs our toolchain supports; this admission filter is a potential selection bias, mitigated by the two additional benchmarks drawn from independent sources.

Comment 1.5.

The paper compares only with fine-tuned CodeT5 and T5Gemma2 variants. The absence of any comparison with modern LLMs is a notable gap. The authors could at least report zero-shot pass rates for state-of-the-art proprietary and open-weight LLMs on the same benchmarks to contextualize the absolute numbers.

[Weaknesses:] *No comparison with modern decoder-only large-scale code models.*

Response.

Thanks for this comment. We added exactly the requested zero-shot comparison, plus a few-shot condition, for three larger open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B) on all four benchmark families (Table 3, p. 26; protocol in the table note). The comparison contextualizes the absolute numbers:

- **SuFu (58 tasks).** None of the three models solves a single task zero-shot, and few-shot prompting reaches at most 8/58, whereas TyFlow-2B solves 21/58. The low-resource language is precisely where the type-guided representation delivers its advantage, and where scale alone does not.
- **Java.** The prompted models solve 33–45 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks, well above the fine-tuned 2B models (TyFlow-2B: 17/8/31); on Java, large decoder-only models are indeed stronger in absolute terms, and we state this plainly, since it is exactly what the comparison is for.
- **A caveat we flag ourselves.** Few-shot prompting brings little or no gain over zero-shot on Java for these models, which suggests they have seen this language extensively and may have been trained on these benchmarks directly; the revision therefore flags a possible data-leakage risk for the Java comparison.

Relative to its fine-tuned baseline T5Gemma2-2B, TyFlow-2B still improves all four benchmarks, which is the controlled comparison the paper’s claims rest on.

Changes made.

- **Added** Table 3 (p. 26): pass@1 of MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks, zero-shot and few-shot, with the prompt protocol in the table note.
- **Added** Sec. 6.2.1 (p. 26): the reading of this comparison, including the data-leakage caveat.

Revised manuscript, Sec. 6.2.1, setup (p. 26) – the setup of the decoder-only comparison

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

Revised manuscript, Sec. 6.2.1, results (p. 26) – the decoder-only results on Java and SuFu

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, whereas TyFlow-2B solves 21 [...]. On Java the conclusion is the opposite: the prompted models solve 33–45 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks, well above the fine-tuned 2B models. Moreover, few-shot prompting on Java brings little or no gain over zero-shot for these models, which suggests that they have seen this language extensively and may even have been trained on these benchmarks directly; we therefore flag a possible data-leakage risk for the Java comparison.

Comment 1.6.

What do the remaining failing programs look like? Are they semantically wrong but well-typed? Do they fail on edge cases? A qualitative analysis of failure modes would significantly strengthen the paper.

Response.

Thanks for this suggestion. We added a full failure-analysis appendix (App. C, p. 42–43). For every test task across the three Java benchmarks (MBJP 67, HumanEval-Java 16, TransCoder-GFG 103), we classify the outcome into one of four mutually exclusive categories, and compare the baseline with TyFlow on each category.

The four categories are:

1. **Solved@1**: the top-1 candidate passes all test cases.
2. **Ranking failure**: a correct candidate exists among the ten, but it is not ranked first.
3. **All-invalid**: no candidate compiles.
4. **Well-typed but wrong**: at least one candidate compiles, but none passes all tests.

How the baseline and TyFlow compare on each category:

- **All-invalid generation essentially disappears.** Under the baseline, 4 MBJP tasks and 2 TransCoder-GFG tasks produce no compilable candidate. Under TyFlow, the counts are 0, 0, and 2 on MBJP, HumanEval-Java, and TransCoder-GFG, respectively; the two residual GFG cases arise from budget exhaustion rather than completed ill-formed output.
- **Solved@1 increases on every benchmark.** MBJP: $9 \rightarrow 17$ (baseline $\rightarrow$ TyFlow). TransCoder-GFG: $20 \rightarrow 31$. HumanEval-Java: $5 \rightarrow 8$. The method adds value beyond reducing compilation errors.
- **Well-typed but wrong becomes the dominant residual mode.** On MBJP, 38 of TyFlow's 50 unsolved tasks produce compilable but incorrect code; on TransCoder-GFG, 53 of 72 do. This is the expected result: TyFlow enforces the implemented type constraints, not semantic correctness.
- **Ranking failures persist.** On MBJP and TransCoder-GFG, 12 and 17 tasks respectively contain a correct candidate in the top ten that is not ranked first. This is a property of beam scoring, not of the representation.

Changes made.

- **Added** App. C (p. 42–43): the four-category taxonomy, Table 8 with counts for the three Java benchmarks, and two representative cases.
- **Added** Sec. 6.2.1 (p. 25): the sentence linking the 220M MBJP result to the taxonomy.

Revised manuscript, App. C (p. 42) – the failure taxonomy and counts

Invalid generation essentially disappears under TyFlow: the all-invalid category drops from 4/0/2 to 0/0/2 tasks, the residual GFG cases being budget exhaustion rather than ill-formed output. Ranking failures persist (12–17 tasks on MBJP and TransCoder-GFG): the constrained search often finds a correct derivation but does not rank it first, a property of beam scoring rather than the representation. The dominant residual mode is well-typed but semantically wrong code: 38/50 unsolved MBJP, 7/8 unsolved HumanEval-Java, and 53/72 unsolved GFG tasks have at least one compilable candidate with incorrect behaviour; this is exactly the boundary of what type-level constraints can guarantee.

Revised manuscript, App. C.2, representative case (p. 43) – the representative semantic failure case

The candidate has the right shape—a loop over indices with a modulo test—but the body never inspects `nums.get(i)` and never assigns `true`; the method returns `false` for every input, while the tests expect `true` for two of the three cases. No typing rule is violated, so no amount of type-level pruning can reject this candidate: the failure is purely semantic.

Response to Reviewer 2

We thank the reviewer for the positive assessment of the theoretical foundation, the framework design, and the ablation study.

Comment 2.1.

W1: Limited evaluation benchmarks. The evaluation uses only a limited number of SuFu test tasks and Java test tasks. These are very small test sets, and no confidence intervals or statistical significance tests are reported. In addition, the SuFu language is highly niche. While this is positioned as a “low-resource” evaluation, it’s unclear how findings generalize beyond this specific setting. There is no evaluation on standard, widely-used benchmarks. While the approach requires language-specific tooling (type checker, parser), the absence of comparison on common benchmarks could limit the paper’s impact and make it harder to contextualize the results.

(Addresses W1) Report confidence intervals or evaluate over multiple random train/test splits to establish statistical stability given the small test sets. Consider adding at least one more widely-used benchmark to help contextualize the results.

Response.

Thanks. We did both things the action item asks for, and we state SuFu’s role explicitly rather than claiming generality from it.

(1) Two widely-used benchmarks. HumanEval-Java and TransCoder-GFG are added with fixed splits of 146/16 and 414/103 tasks (the quoted passage describes them); the Java evaluation grows from 67 to 186 test tasks across three independent sources. We did not add multiple random splits (the SuFu and added Java splits are fixed by construction); instead we report the intervals and paired tests of the first option.

(2) Statistical stability. Rather than reporting point estimates on small sets, App. D reports 95% intervals and exact paired tests for every comparison, at both scales, with the three Java benchmarks pooled into a 186-task analysis. On the pooled set all four metrics are significant, and the appendix identifies explicitly where the evidence remains underpowered (220M Java, 2B SuFu) instead of implying significance (Comment 1.3).

(3) SuFu’s role. SuFu stays in the evaluation as the low-resource case, and the revision says so: it is the setting in which the type-guided representation helps most and in which larger decoder-only models fail entirely zero-shot (Comment 1.5), while the Java results provide the generalization evidence on widely used benchmarks.

Changes made.

- **Added** Sec. 6.1.1 (p. 24): HumanEval-Java and TransCoder-GFG with their fixed splits (146/16 and 414/103).
- **Added** App. B (p. 37–40): the construction of the two benchmarks and Table 7.
- **Added** App. D (p. 44): the pooled 186-task Java analysis.

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added benchmarks and their splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java, and TransCoder-GFG collects the Java programs released with TransCoder; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, App. B (p. 40, Table 7) – split statistics of the three Java benchmarks

HumanEval-Java and TransCoder-GFG problems are short single-method snippets (median 12 LoC, 21–35-word prompts), whereas MBJP is the densest and most example-rich benchmark (median 27 LoC, prompt with three executable example cases and a gold solution about twice as long as the additional benchmarks).

Comment 2.2.

W2: Scalability concerns are insufficiently addressed.

The paper does not discuss how the synthesis derivation tree scales with program complexity. Real-world programs could have deep type derivation trees with polymorphism, subtyping, type inference, and overloading resolution, but the Java subset avoids most of these. It is also unclear what runtime overhead the synthesis system imposes during inference, as the dynamic re-encoding at each step and the unification/constraint-checking machinery introduce costs that are never quantified.

(Addresses W2) Report runtime inference time comparisons between TyFlow and the baselines. Discuss, even qualitatively, how the synthesis system would scale to richer type features such as polymorphism with variance, subtyping hierarchies, or ownership types. Report how frequently beam search exhausts all candidates due to pruning and what the fallback behavior is.

Response.

Thanks. We address each part of the action item.

(1) How the synthesis tree scales with program complexity. The synthesis derivation tree is represented as a token sequence whose length is roughly proportional to the program’s own token length. On SuFu, the average TyFlow decision sequence is 410 tokens, comparable to the average program’s 405 code tokens; on Java, 129 tokens vs. 142 program tokens. The derivation tree therefore grows linearly with the program, not exponentially.

(2) Runtime overhead, measured. RQ2 now times the same 220M model in the four component

configurations, reporting per-step ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms) and per-task totals (-6.38% / $+94.00\%$ / $+173.88\%$, $2.74\times$ the unconstrained base); the quoted paragraph explains where each cost comes from and why syntactic pruning even lowers the per-task total.

(3) Scaling to richer type features. This issue is answered in detail in [Comment 1.1](#). In short, richer type features add constraints, state, and rule choices. They do not change the mapping from typing rules to synthesis rules when their checks can use first-order terms and separate solvers, but they do increase implementation work, solver cost, and search. Our experiments do not show that these costs scale to full production languages.

(4) Beam exhaustion and fallback. The instrumented run reports zero tasks that dead-end under beam 10. By design, if beam search ever ran out of candidates, the task would be counted as a failure. In practice this never occurred across all 58 SuFu tasks and all 186 Java tasks.

Changes made.

- **Added** Sec. 6.2.2 (p. 27): the runtime measurements and the Time / ms-per-step columns of Table 4.
- **Added** Sec. 6.2.1 (p. 26): the discussion of how the synthesis derivation tree scales with program complexity.
- **Added** Sec. 6.2.2 (p. 27): the beam-exhaustion observation (no task dead-ends under beam 10).
- **Added** Sec. 6.3 (p. 30): the limitations on type-system expressiveness.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 27) – the runtime measurements

Each component increases the per-step cost ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms). Syntactic pruning adds little (roughly $+4\%$); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal. The per-task totals move by -6.38% / $+94.00\%$ / $+173.88\%$ ($2.74\times$ base). [...]]

Revised manuscript, Sec. 6.2.2 (p. 27) – the beam-exhaustion observation

A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [...]] With beam 10, this number averages 7.97. [...]] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – the scope of richer type-system extensions

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. The current solver cannot handle a typing rule that requires higher-order unification, as discussed in Sec. 3. Our experiments do not test either type of extension. We leave them for future work.

Comment 2.3.

W3: Comparison with modern LLMs is absent. The baselines are CodeT5-220M and T5Gemma2-2B, and both are relatively small encoder-decoder models. There is no comparison with decoder-only models which are much more common in today's community. Additionally, no model larger than 2B parameters is evaluated. While the paper demonstrates clear improvements over its chosen baselines, it remains an open question whether larger models might already mitigate type errors sufficiently through scale alone. This would contextualize the practical value of the proposed approach.

(Addresses W3) Include at least a zero-shot or few-shot evaluation of a modern decoder-only model on the same benchmarks. Additionally, discuss whether the core ideas could be adapted to decoder-only architectures, even if full integration is left to future work.

Response.

Thanks. We address both parts of the action item with new experiments and new text.

(1) The comparison with larger decoder-only models. We evaluate three open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B; 7B–30B parameters) on all four benchmarks, zero-shot and few-shot, with one greedy candidate per task (Table 3, p. 26). The results answer the reviewer's open question directly: on Java, larger models do achieve higher absolute performance (33–45 of 67 MBJP tasks, well above TyFlow-2B's 17/67), confirming that scale helps on well-resourced languages. On SuFu, however, scale alone is insufficient: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8/58, against 21/58 for TyFlow-2B. Type errors, in other words, are not eliminated by scale alone in the setting our method targets.

(2) The architectural discussion. The Limitations subsection now discusses what a decoder-only adaptation would involve: the representation transfers unchanged, but the dynamic typing context would need to be prefilled and removed at every step, so its key-value entries are recomputed each step and never reused, making the computation awkward to schedule. We state this as future work rather than asserting feasibility.

Changes made.

- **Added** Table 3 (p. 26) and Sec. 6.2.1 (p. 26): the decoder-only comparison.
- **Added** Sec. 6.3, "Dependence on the encoder-decoder architecture" (p. 29): what a decoder-only adaptation requires.

Revised manuscript, Sec. 6.2.1 (p. 26) – the decoder-only comparison

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot [...] On Java the conclusion is the opposite: the prompted models solve 33–45 of 67 MBJP tasks [...] well above the fine-tuned 2B models.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29) – the decoder-only adaptation limitation

The representation itself carries over to a decoder-only model unchanged; what changes is the architecture, and with it the cost of the dynamic typing context. In a decoder-only model, supplying the context would mean prefilling the current typing context after the derivation prefix at every step and removing it again before the next step, so its key-value entries are recomputed at every step and never reused by later steps, and the computation is awkward to schedule. We leave this adaptation to future work.

Comment 2.4.

W4: The CHC generality claim is overclaimed relative to the evidence. The paper repeatedly emphasizes that CHCs can encode “any Turing-computable function” and that the framework generalizes to “arbitrary logical constraints.” However, the evaluation only covers type correctness for two languages with relatively simple type systems. No evidence is provided for richer constraints. The gap between the theoretical generality and empirical validation is large.

(Addresses W4) Either provide a case study demonstrating a constraint richer than basic type correctness (e.g., ownership, effects, refinement types), or explicitly scope the generality claims to what is empirically demonstrated and acknowledge the gap as a limitation.

Response.

Thanks. We agree that the submitted wording overclaimed. We took the second option and scoped the claim everywhere it appeared (introduction, related work, conclusion), separating what the framework is, what this paper evaluates, and what remains future work. We did not produce the requested case study with an additional constraint family, since such evidence is beyond the scope of this revision; the gap is acknowledged as a limitation instead. The empirical scope of the revision is two languages, four benchmark families, 58 SuFu and 186 Java test tasks, with type correctness as the only evaluated constraint family.

Changes made.

- **Scope narrowed** Sec. 1 (p. 4), Sec. 7 (p. 30), and Sec. 8 (p. 31): the CHC generality claims now state that the framework is instantiated and evaluated only for type correctness in two languages.

Revised manuscript, Sec. 1 (p. 4) – the scoped CHC claim in the introduction

Notably, all proof rules in our system are formulated as constrained Horn clauses (see Sec. 3.1.1). This representation not only subsumes the prior work on syntactic correctness but also accommodates a broader class of Turing-computable specifications. While this paper instantiates and evaluates the framework only for type correctness in two languages, the synthesis construction itself is defined at the CHC level and is in principle applicable to other decidable constraint families; we do not evaluate such extensions empirically.

Revised manuscript, Sec. 7 (p. 30) – the scoped CHC claim in related work

Grammar-based encoding can be viewed as a special case of our framework when grammar rules are expressed as CHCs. Our implementation and evaluation exercise this generality on type-correctness constraints in two languages; other CHC-representable constraint families remain a formal possibility that we do not evaluate.

Revised manuscript, Sec. 8 (p. 31) – the scoped CHC claim in the conclusion

Beyond type correctness, TyFlow’s CHC foundation suggests a route toward richer constraint families, such as safety verification and the generation of other structural data; instantiating and evaluating these extensions remains future work.

Response to Reviewer 3

We thank the reviewer for the positive assessment of the typing-based extension of grammar-guided generation and of the rigor of the isomorphism between type derivations and synthesis derivations.

Comment 3.1.

Missing comparisons against token-level constrained decoding works such as SynCode and Copiloting the Copilots. Those works also guide LLM generation, but they are better than rejection sampling because they guide the LLM at the token level. To demonstrate the robustness of TyFlow, comparisons against those works are important.

Response.

Thanks for this suggestion. We compared TyFlow with rejection sampling, SynCode, Repilot (*Copiloting the Copilots*), and iterative compiler repair on MBJP, all running on the same frozen checkpoint under the same sampling protocol. The question is whether these external mechanisms, which operate only on the model’s output at decoding time, can match internalizing the type constraints into the model itself. The results show they cannot: all four filter, mask, or repair candidates without changing the model’s own distribution over programs, whereas TyFlow changes that distribution by encoding the typing rules into the model.

SynCode constrains decoding at each step by masking tokens that would violate the grammar, so every output is syntactically well-formed. It reduces the compilation-error rate (26.12% → 21.34%) but leaves the solved sets unchanged: constraining syntax does not help with semantic correctness, because a well-formed program can still compute the wrong result.

Repilot is the method proposed in *Copiloting the Copilots*. It pairs the LM with the Eclipse JDT completion engine, pruning tokens that the engine cannot legally complete. In our run its pruning rarely fires, for two reasons. First, following the original implementation, punctuation and keyword tokens are accepted without querying the engine, so a large share of the model’s token proposals are never checked at all. Second, when the engine is queried, it is queried at an incomplete position, where its completion list is permissive: JDT can legally continue almost any partial prefix, so few proposals are actually rejected. Repilot therefore behaves close to unconstrained decoding: compilation errors barely move (26.12% → 25.97%), and the solved sets are unchanged.

Rejection sampling is the only external method that changes any outcome, adding three solved tasks at pass@10 and one at pass@1, but neither difference is statistically significant.

TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with only 3/670 compilation errors, the best result among all compared methods. TyFlow enforces the modeled type constraints during generation and reduces CER to 0.45%, rather than filtering or repairing what the model

already produces.

Changes made.

- **Rewrote** Sec. 6.2.3 (p. 28): RQ3 is now a five-method comparison under one frozen checkpoint, and Table 5 was replaced accordingly.
- **Added** Sec. 6.2.3 (p. 28): the shared-protocol statement, the reconciliation of the control with Table 2, per-method descriptions, and the paired tests.

Revised manuscript, Sec. 6.2.3 (p. 28) – the five-method setup and shared protocol

On MBJP, four such methods are applied on top of the same frozen T5Gemma2-2B checkpoint selected in RQ1, adding their own pruning or repair mechanisms on top: *rejection sampling*, *SynCode*, *Repilot*(*Copiloting the Copilots*), and *iterative compiler repair*. All methods are evaluated on the same 67 tasks with 10 candidates each. Because rejection sampling and repair need diverse draws rather than beam output, the ordinary control uses the temperature-sampling decoding protocol (temperature 0.8, top- p 0.95), so its scores (14.93/23.88) differ from the beam-10 row of Tab. 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

Revised manuscript, Sec. 6.2.3 (p. 28) – the SynCode and Repilot mechanisms

SynCode constrains decoding itself: at each step it masks the tokens that would violate a given grammar, so every output is syntactically well-formed. It is not originally implemented for Java, so we adapt it to the grammar of our Java subset. Repilot pairs the LM with the Eclipse JDT completion engine [...] Repilot’s pruning rarely fires: following the original Repilot implementation, punctuation and keyword tokens are accepted without querying the engine, and the completion list at an incomplete position is permissive, so compilation errors barely move (26.12% → 25.97%).

Revised manuscript, Sec. 6.2.3 (p. 28) – the outcome and the Type Explicitness conclusion

None of the four external baselines significantly improves functional correctness over the ordinary control: rejection sampling adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10; McNemar $p = 1.00$ and $p = 0.25$); SynCode, Repilot, and iterative repair leave the solved sets exactly unchanged (10/67 and 16/67) [...] TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with 3 of 670 candidates failing to compile, the best of all compared methods. External reliability mechanisms (post-hoc filtering, token-level grammar masking, completion-engine repair, compiler-feedback regeneration) never change the model’s own preference and distribution over output tokens, whereas TyFlow explicitly encodes the typing rules into the model and helps it better learn the program distribution, demonstrating the importance of the Type Explicitness property in Sec. 1.

Comment 3.2.

Missing comparison against iterative refinement approaches: Compared to rejection sampling, iterative refinement approaches regenerate the program with feedback (in this case, the typing failure feedback), which is strictly stronger than rejection sampling as it incorporates typing failure feedback. To demonstrate TyFlow’s robustness, the authors should compare it with this line of work.

Response.

Thanks for this comment. Iterative compiler repair is now one of the five compared methods: javac diagnostics are fed back to the model, which regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics.

Repair does improve compilability (compile errors fall from 175 to 116 of 670 candidates) but adds no solved task: the solved sets remain 10/67 at pass@1 and 16/67 at pass@10, whereas rejection sampling in the same run adds three solved tasks at pass@10 and one at pass@1 — so feedback-based regeneration is not strictly stronger than rejection sampling on this benchmark. Note that rejection sampling’s own gain is itself not statistically significant (App. D) — neither external method demonstrably improves functional correctness over the ordinary control.

The theoretical advantage of feedback does not show up here because of the experimental setup, in two ways. First, the budgets are not equalized, since each method keeps the protocol of its own line of work: rejection sampling draws up to 100 candidates per task, while repair works from ten sampled candidates plus at most two feedback rounds. Second, the agent is a 2B model, which is not yet good at turning diagnostics into correct rewrites; a larger agent might extract more from the feedback, but swapping it in would change the model rather than the mechanism under comparison. And the deeper reason is independent of budget: none of the 59 repaired candidates passed its tests, because the dominant residual failure is well-typed but semantically wrong (App. C), exactly what compile diagnostics cannot see. RQ3 concludes: feedback changes what compiles, not the model’s distribution over semantically meaningful programs, whereas TyFlow encodes the typing rules into the model itself.

Changes made.

- **Added** Sec. 6.2.3 (p. 28): the repair protocol (javac feedback, at most two rounds, un-fine-tuned agent), its measured outcome, and the closing discussion.

Revised manuscript, Sec. 6.2.3 (p. 28) – the iterative-repair protocol

Iterative compiler repair feeds javac diagnostics back to the model and regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics.

Revised manuscript, Sec. 6.2.3 (p. 28) – the repair outcome vs. rejection sampling

[...] iterative repair repairs 59 candidates’ compilability (compile errors 175→116 of 670) yet leaves the solved sets unchanged.

Comment 3.3.

The TyFlow system requires multiple LLM calls to obtain all synthesis decisions. To show the practicality of TyFlow, it is important to provide an analysis of the system's cost for generating the program.

Response.

Thanks for this question. We first clarify the base case, then explain what our method adds on top of it.

(1) The base case is a single encoder–decoder call. The system is not a multi-LLM pipeline: the encoder processes the prompt once, and the decoder generates the output token by token, reusing the encoder's representation throughout. There are no repeated LLM calls to separate models.

(2) What our method adds on top of this base. During decoding, we add syntactic pruning (checking each generated token against the grammar), type pruning (checking completed AST fields against the type system), and a dynamic typing context (re-encoding the synthesis goal as it evolves). Each of these adds moderate per-step cost, but the encoder's computation is fully reused, so the total does not grow proportionally.

(3) The measured cost. RQ2 now times the same 220M model in the four component configurations, reporting per-step ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms) and per-task totals ($5.70 / 5.34 / 11.07 / 15.62$ s, i.e. $2.74\times$ the unconstrained base). Each increment has a stated cause (per-step type checking; re-encoding the evolving synthesis goal). For the compared external methods, the RQ3 setup states each method's computational budget rather than leaving costs unstated.

Changes made.

- **Added** Sec. 6.2.2 (p. 27): per-step and per-task runtime for the four configurations (Table 4).
- **Added** Sec. 6.2.3 (p. 28): the computational budget of each compared external method.

Revised manuscript, Sec. 6.2.3 (p. 28) – the rejection-sampling budget

Rejection sampling repeatedly draws candidates and keeps those that pass Java compilation, for up to ten ten-draw rounds (100 candidates) per task.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 27) – the runtime measurements

Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal.

Comment 3.4.

Limited Java Performance: Although TyFlow brings significant improvement on SuFu, its performance boost on Java is marginal (from 10.45% to 11.94%).

Response.

Thanks for this observation. This is the same question as [Comment 1.4](#), and the answer is the different starting points of the two baselines. On SuFu, the baseline is a model for a low-resource language with many type errors, so the constraints have a lot to fix and pass@1 rises substantially (24.14% → 37.93%). On Java, the baseline has already been trained extensively on the language, so there is little room for pass@1 to improve. Once the evaluation moves to the 2B scale with the extended test sets, the Java gains become much larger, and on the pooled 186-task analysis all four differences are significant ([Comment 1.3](#)); the Java conclusions rest on those results.

Changes made.

- **Added** Sec. 6.2.1 (p. 25): the paragraph explaining why the 220M MBJP gain is the smallest.
- **Added** App. B (p. 40) and App. D (p. 44): the baseline protocol and the pooled significance analysis behind these rows.

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java results at both scales

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises from 24.14% to 37.93% (SuFu) and from 10.45% to 11.94% (MBJP) at 220M, and from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, Sec. 6.2.1 (p. 25) – why the 220M MBJP gain is the smallest

The 220M MBJP pass@1 gain is the smallest because there the type constraints mainly remove uncompileable candidates; the benefit shows up in pass@10 and CER instead, consistent with the failure analysis in Appendix C (well-typed but wrong is the dominant residual mode).

Comment 3.5.

Limited model choices: The TyFlow system was mainly evaluated on small models. Evaluation on larger models (decoder-only, billions of parameters) is unexplored. The authors should consider extending the evaluation to larger models.

Response.

Thanks for this comment. This question is already answered in detail under [Comment 1.5](#) (the added zero- and few-shot comparison, and its data-leakage caveat) and [Comment 2.3](#) (the same results read against the open question of larger models, plus the architectural discussion); we restate the analysis and conclusions here so that they can be read on their own.

The added comparison. Three open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B; 7B–30B parameters) are evaluated on all four benchmarks, zero- and few-shot, under a separate prompt-based protocol kept distinct from the trained results (Table 3, p. 26); the evaluation now spans five model scales.

The two-sided analysis. On Java the decoder-only models are clearly stronger in absolute terms than the fine-tuned 2B models, and the revision says so plainly, since this is exactly what such a comparison is for; it also flags a possible data-leakage risk for the Java comparison, because few-shot prompting brings these models little or no gain over zero-shot. On SuFu, the low-resource setting the method targets, none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, against 21 of 58 for TyFlow-2B.

The conclusions. Scale helps on well-resourced languages, but it does not eliminate type errors in the setting the method targets, so it does not substitute for the type-guided representation. Whether TyFlow itself can be adapted to decoder-only architectures is discussed in the Limitations (Sec. 6.3), including what the adaptation would cost; that adaptation is left to future work.

Changes made.

- **Added** Table 3 (p. 26): the three decoder-only models on all four benchmarks.
- **Added** Sec. 6.3 (p. 29): the architectural limitation for a decoder-only adaptation.

Revised manuscript, Sec. 6.2.1, Table 3 and its note (p. 26) – the decoder-only comparison and its protocol

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task [. . .] Cell entries are solved tasks at pass@1 for all models; the 0-shot condition uses the same prompt as the two fine-tuned models.

Comment 3.6.

Generalizability concern: TyFlow relies strongly on the typing system of the language, but how would it work on a dynamically typed language such as Python? Thus, the author should acknowledge this in the threat to validity to make this paper more robust.

Response.

Thanks for this comment. A dedicated limitations paragraph now states the obstacle: TyFlow needs the type derivation to be computable from the program text, which is what makes training data automatic in the two evaluated languages (290 SuFu programs, 608 Java training tasks); in a dynamically typed language the derivation depends on control flow and runtime information, so decision sequences cannot be extracted from existing code, the Data Usability property no longer holds, and adaptation is left to future work.

Changes made.

- **Added** Sec. 6.3, “Dynamically typed languages” (p. 30): the limitation and why the training-data pipeline does not carry over.

Revised manuscript, Sec. 1, “Data Usability” (p. 3) – the Data Usability property

The decision sequences of S and source code should be automatically and bidirectionally convertible, enabling both the extraction of decision sequences from existing programs and the reconstruction of programs from generated decision sequences.

Revised manuscript, Sec. 6.3, “Dynamically typed languages” (p. 30) – the dynamic-language limitation

The current design of TyFlow relies on a type system in which the type derivation of a program can be computed automatically from the program text, so that existing code can be converted into our decision-sequence format. Dynamically typed languages such as Python still have typing rules to a degree, but the type derivation cannot be read off the program text alone, since types depend on control flow and runtime information. [. . .] Parsing and adapting such languages is left to future work.

Other Changes beyond the Reviewers' Requests

- **Re-trained results.** The results in the submitted version were produced in 2025 in a different environment, and those checkpoints and evaluation records are not available on the server used for this revision; every model reported here was therefore re-trained with the same framework and the same code, and all evaluations were re-run. The new run supports the conclusions of the submitted version; the differences are small (most values move by less than two percentage points; the largest is the 2B SuFu row, where the re-measured baseline is 25.86% (submitted: 29.31%) and the reported TyFlow-2B value is 36.21% (submitted: 43.10%); TyFlow still exceeds the baseline on all four metrics); the intervals and paired tests of Appendix D are computed on the new runs, so the significance statements apply to the numbers printed in the tables.
- **Errata.** Algorithm 1 now composes substitutions as $\sigma_{i-1} \dots \sigma_0 \sigma$; the statement after Lemma 2 reads $P(\theta(\bar{i}))$; and the proof of Lemma 4 defines $\theta_c = \sigma_n \dots \sigma_0 \sigma$. The method and the theorems are unchanged.
- **FSP definition.** Sec. 6.1.3 (p. 24): a task with no correct candidate contributes the candidate-list length, a convention the submitted text left implicit.

We hope that the revision addresses all concerns and look forward to the reviewers' feedback.

Sincerely,
The Authors